Computer Science 2

CS-2 Project

Documentation of Self-record Attendance System

Members

Alve, Orejola

Documentation **2**
Problem Statement: 2
Project Objectives: 3
Planned Inputs and Outputs: 4
Process Flowchart: (Image Link for Flowchart) 6
File Structure: 8
Technologies/Tools Used: 9
Known Issues: 9
User Guide: 10
Introduction 10
Methodology **12**
Methodology 12
1. Implementation of Features: 12
2. Technologies Used (with justification) 15
2.1 Python 15
2.2 Tkinter 15
2.3 bcrypt 16
2.4 SQLite 16
2.5 tkCalendar 17
2.6 pandas 17
2.7 pytest 17
Backend-frontend Communication 17
3.1 Tkinter (Frontend) 17
3.2 Python (Backend) 18
3.3 SQLite (Storage) 18
4. Key Design Decisions or Trade-offs 18
4.1 Pop-up validation messages 19
4.2 Database Usage (SQLite) 19
4.3 Exception-based attendance marking 19
5. References to Ethical Considerations in Programming Choices 20
5.1 Data protection: 20
5.2 Pop-up Messages: 20

Documentation

Problem Statement:

Regarding attendance, both manual and automated tracking methods are still used. While they are reasonably efficient, issues can still happen. Manual attendance depends on one or more people to check students in, which can be prone to errors or delays. On the other hand, automated systems such as QR scanners and on-campus terminals are faster and more efficient, but each terminal can serve only one student at a time. Even with multiple terminals, lines of students are still likely to form.

Additional Info 1: On November 21, 2025, during Noise vs. Voice Talk Attendance

During attendance, students sign their names next to it in a list. This works, but because there are many students, the process can take time, especially if there are delays from technical issues or late arrivals, which can throw off the schedule.

Additional Info 2: Role of Homeroom / Class Adviser in Student's attendance (Based on the PSHSS CIM 7.0 Student's Attendance V2_Rev1.pdf)

The Homeroom/Class adviser checks and combines students' attendance to update their report cards (and reports to SCO if needed). They also need to submit two copies of the filled-out Record on Attendance and Punctuality to the Homeroom Unit Coordinator. Since students' attendance is needed for many days, these steps can become tedious.

For this automated attendance system project, the goal is to create a system that fixes these issues, mainly by allowing teachers to create attendance during class periods, and ask them to input any exceptions to the assumption that all students are present there (i.e, execution-based inputting).

Project Objectives:

• Allow teachers to record attendance sessions through exception-based inputs for student punctuality.

• Provide a transparent and assistive attendance tool that may help with the current limitations of traditional and semi-automated attendance systems.

Feature List:

• Sign up, Login, Logout feature (Account Authentication)

• Role-based system attendance (Teachers, Students)

• Attendance features: Create Attendance, Attendance Dashboard.

• Data export of attendance records to CSV (Comma Separated Values)

• Pop-up messages for user actions in connection with event-driven GUI

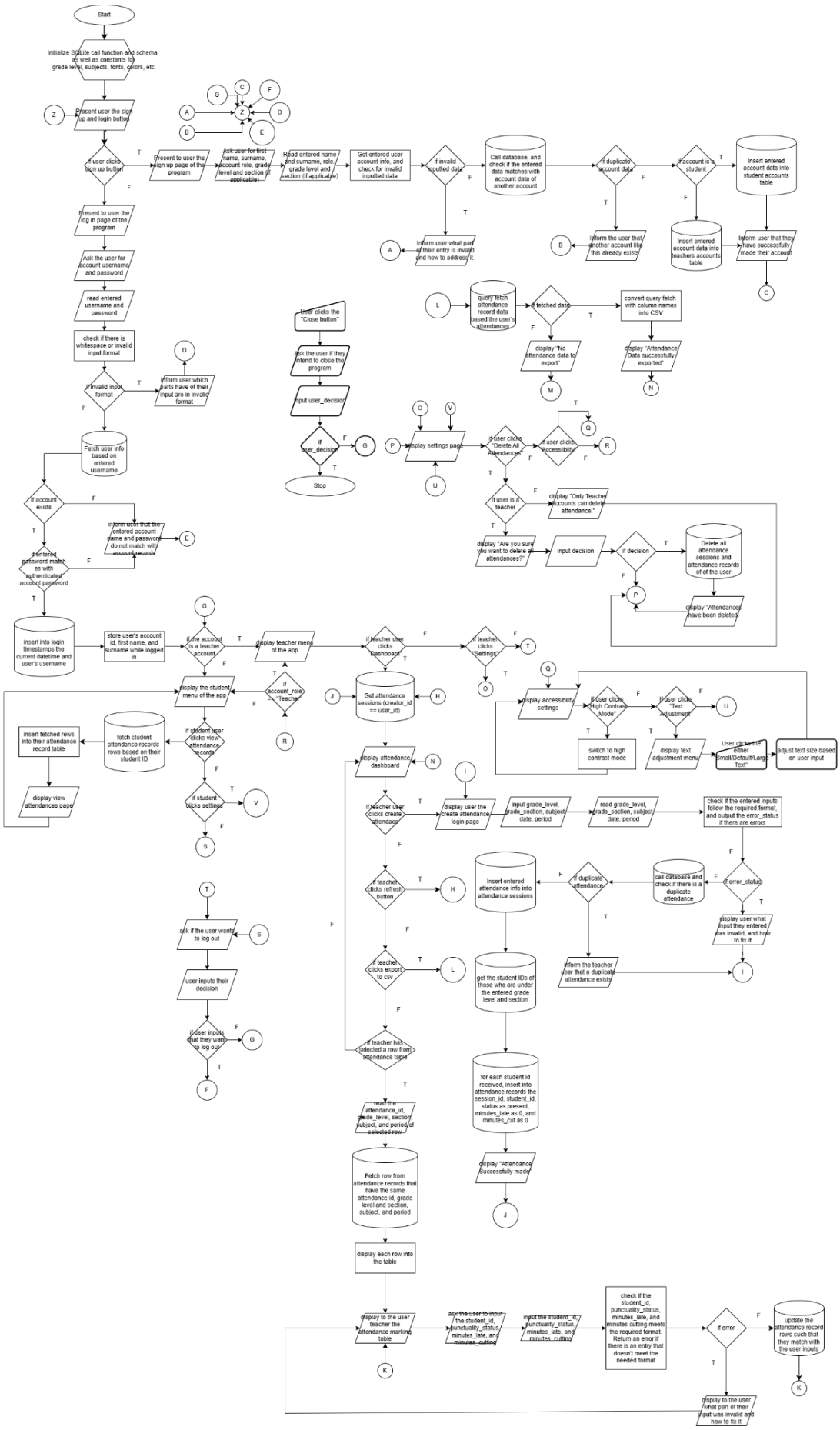
Planned Inputs and Outputs:

Input	Output
User opens Program	The program prompts the user to either create an account or log in to their account.
User opens the create account page	The program gives the user a form that the user needs to fill out to make an account. This includes their first name and surname, username, account role type, grade level and section (if their account role is a student), and account password.
User clicks “Create Account.”	The program will inform the user if there is an error in their inputs (e.g, no account role selected, account password <10 characters, empty inputs, grade section not associated with the grade level, such as 7 - Rubidium). Duplicate accounts are an error. If there are no errors, the program will inform the user that their account has been successfully created.
User opens the login page	The program prompts the user to enter their account username and its password.
User clicks “Login.”	The program reads the inputs and checks if the entered username and password are in the database and match with each other. If not, then the program informs the user about

	this. If the login is successful, the program will show different menus depending on the account role type. For this case, teacher accounts can access the attendance dashboard, while student accounts can access the “view attendance records”. Both accounts have access to the settings menu.
Teacher Account Clicks Attendance Dashboard.	The program prompts the user with the “create attendance button”, a refresh page button, and a button to export attendance data. The program also shows the list of attendances the user has made.
Teacher Account Clicks Create Attendance	The program prompts them to fill out a form to make an attendance record. The form asks for the grade level and section, subject, date, and the period the class starts at.
The Teacher Account creates an attendance after filling out the “create attendance” form	The program reads through the teacher’s inputs and checks if: ● Grade level and section have an entry, and the grade section is under the grade level ● The subject has been entered and is part of the grade level. ● Date has an entry, is in MM-DD-YYYY, and is not a weekend. ● The period has an entry, and it is a number integer from 1 to 10. The program informs the user if any of these “input checks” fail. (Duplicate attendances are an error) If no error, the program saves the entered info into the database.
The teacher refreshes the teacher dashboard.	The program clears the table in the attendance dashboard, gets back all the info (new or otherwise) from the attendance sessions stored by the user, and puts the data back into the attendance dashboard table.
The teacher exports attendance data	The program reads through the info about their made attendance sessions and records, and makes a CSV file about that data. This will not make a CSV file if the user has not

	yet made an attendance, nor have they had any attendance data made.
The teacher clicks an attendance row in the attendance dashboard table.	The teacher user is redirected to the attendance marking frame, where they can edit their attendance records in case some students are either tardy, cutting, or absent from their class.
Student Account Clicks “View Attendance Records”	The program displays a table containing their attendance records, specifically the attendance ID, date, subject, start period, and status.
The user (student or teacher) opens the settings menu.	The program opens a menu that has the option to delete attendance and go to accessibility settings.
User clicks “delete all attendances” in settings	The user checks first if they have a teacher account and confirms if they intend to delete the attendance records they made. If they are a teacher and they confirm that they wish to delete all their made attendance records, then the program gets the user’s id, looks through attendance sessions and records, and deletes the attendance sessions and their attendance recordings.
The user clicks “Log out” in the main menu	The program asks first if they intend to log out before doing so.

Process Flowchart: (Image Link for Flowchart)


Start
Initialize SQLite call function and schema, as well as constants to grade level, subjects, fonts, colors, etc.
Present user the sign up and login button
if user clicks sign up button
Present to user the sign up page of the program
Ask user for first name, surname, account role, grade level and section (if applicable)
Read entered name and surname, role, grade level and section (if applicable)
Get entered user account info, and check for invalid inputted data
if invalid inputted data
Inform user what part of their entry is invalid and how to address it.
Call database, and check if the entered data matches with account data of another account
if duplicate account data
Inform the user that another account like this already exists
if account is a student
Insert entered account data into teachers accounts table
Insert entered account data into student accounts table
Inform user that they have successfully made their account
Present to user the log in page of the program
Ask the user for account username and password
read entered username and password
check if there is whitespace or invalid input format
if invalid input format
Inform user which parts have of their input are in invalid format
Fetch user info based on entered username
if account exists
inform user that the entered account name and password do not match with account record
if entered password matches with authenticated account password
insert into login timestamps the current datetime and user's username
store user's account id, first name, and surname while logged in
if the account is a teacher account
display teacher menu of the app
if teacher user clicks Dashboard
if teacher clicks Settings
display the student menu of the app
if account_role == "Teacher"
Get attendance sessions (creator_id == user_id)
display attendance dashboard
if student user clicks view attendance records
fetch student attendance records rows based on their student ID
insert fetched rows into their attendance record table
display view attendances page
if student clicks settings
ask if the user wants to log out
user inputs their decision
if user inputs that they want to log out
User clicks the "Close button"
ask the user if they intend to close the program
input user_decision
if user_decision
Stop
display settings page
if user clicks "Delete All Attendances"
if user clicks "Accessibility"
if user is a teacher
display "Only Teacher Accounts can delete attendance."
display "Are you sure you want to delete all attendances?"
input decision
if decision
Delete all attendance sessions and attendance records of the user
display "Attendances have been deleted"
query fetch attendance record data based the user's attendances
if fetched data
convert query fetch with column names into CSV
display "No attendance data to export"
display "Attendance Data successfully exported"
display accessibility settings
if user clicks "High Contrast Mode"
if user clicks "Text Adjustment"
switch to high contrast mode
display text adjustment menu
User clicks the either Small/Default/Large Text
adjust text size based on user input
if teacher user clicks create attendance
display user the create attendance login page
input grade_level, grade_section, subject, date, period
read grade_level, grade_section, subject, date, period
check if the entered inputs follow the required format, and output the error_status if there are errors
if error_status
display user what input they entered was invalid, and how to fix it
call database and check if there is a duplicate attendance
if duplicate attendance
Inform the teacher user that a duplicate attendance exists
Insert entered attendance info into attendance sessions
get the student IDs of those who are under the entered grade level and section
for each student id received, insert into attendance records the session_id, student_id, status as present, minutes_late as 0, and minutes_cut as 0
display "Attendance Successfully made"
if teacher refresh button
if teacher clicks export to csv
if teacher has selected a row from attendance table
read the attendance_id, grade_level, section, subject, and period of selected row
Fetch row from attendance records that have the same attendance id, grade level and section, subject, and period
display each row into the table
display to the user teacher the attendance marking table
ask the user to input the student_id, punctuality_status, minutes_late, and minutes_cutting
input the student_id, punctuality_status, minutes_late, and minutes_cutting
check if the student_id, punctuality_status, minutes_late, and minutes cutting meets the required format. Return an error if there is an entry that doesn't meet the needed format
if error
update the attendance record rows such that they match with the user inputs
display to the user what part of their input was invalid and how to fix it

File Structure:

```
attendance_app/
    main.py
    utils/
        validators.py
        helpers.py
        constants.py
        __init__.py
    ui/
        handlers.py
        frames.py
        components.py
        __init__.py
    services/
        attendance_service.py
        auth_service.py
        __init__.py
    db/
        attendance_repo.py
        db.py
        user_repo.py
        __init__.py
```

Technologies/Tools Used:

- Python
- Tkinter (tk interface)
- SQLite
- Bcrypt
- tkCalendar
- pandas (i.e, the external, open-source library that is separate from Python's standard library
- pytest

Known Issues:

- Tight Coupling within the "main(dot)py" file. For context, coupling, from a computer programming point of view, is the dependency that certain code blocks have on other code blocks (Coupling and cohesion - software engineering, 2025).
- No admin role - Currently, there are only roles for students and teachers. As a result, there would be little to no oversight of actions happening in the app (The only way to oversee users' actions is by accessing the database itself and looking through the timestamps.)
- Limited Scalability concerning other Campuses - The current labels for grade and section are only oriented for the PSHS Eastern Visayas Campus in Leyte, which can cause scaling issues if:
    - Section Names change in PSHS-EVC
    - Another campus uses the code app

User Guide:

Introduction

APACE (Assistive Project Attendance Considering Efficiency) is a code program/app that allows teacher to manage their attendance sessions, and allows students to view their attendance records. APACE mainly aims to, as the project title suggests, act as an assistive tool for helping teachers manage attendance records to reduce the inefficiencies noticed in manual attendance recording.

Getting Started:

How to install and open the program: [README - How to run the program]

Upon opening the program, you are shown the start menu of the app, containing buttons to either create an account or log in to an existing one.

Creating an account:

1. In the start menu, click “Sign Up”. You will be shown a sign-up form containing the required info needed to fill out your account details.
2. In the sign-up form, enter your:
    a. Name
    b. Surname
    c. Account Role
    d. Grade & Section (If you are making a student account)
    e. Password
3. Click “Create Account”. Note that all fields shown to you must be filled out. Additionally, your account password should have at least 10 characters, and no duplicate accounts can be made

Logging In:

1. Enter account username and password
2. Click Login. Depending on your account role, you would either be directed to the teacher menu or the student menu of the code program

Teacher Account Features:

Teacher accounts can access the attendance dashboard, which consists of a table of the attendance you made, and three buttons: “Create Attendance”, “Refresh,” and “Export data.”

Create Attendance:

Steps:

1. Click “Create Attendance” in the attendance dashboard. You will be shown a form containing the required entries for attendance creation.
2. Enter:
    a. Grade & Section
    b. Subject
    c. Date (Format - MM-DD-YYYY)
    d. Period
3. Submit form

How to access and manage attendance made:

1. In the attendance table of the dashboard, click the attendance you wish to manage.
2. You will be shown the attendance marking frame, consisting of session info, the list of students,
3. Additionally, there will be a mini-form for marking students’ attendance in which you can enter the student ID, punctuality status, and minutes tardy or cutting (if applicable).

Exporting Data: This generates a CSV file of the attendance records of your attendance sessions

Student Features:

Viewing Attendance - Steps:

1. Click “View Attendance Records” in the student menu. In there, you will find a table consisting of the attendance ID, date, subject, period, and your punctuality status

Settings:

- Delete All Attendances - Deletes All Attendance Sessions and Records you have. Ensure you have exported your data first before doing so. **This feature is for teachers only.**
- Accessibility Settings (High Contrast Mode, Text Size Adjustment)

Logging Out:

1. Click the “Log Out” button in your menu.
2. A pop-up will occur to confirm if you want to log out of your account. Click yes or no.

APA References:

1. Coupling and cohesion - software engineering. (2025, April 24). Coupling and Cohesion - software engineering. GeeksforGeeks.

Methodology

Methodology

1. Implementation of Features:

What? (The Feature)	Why? (Rationale)	How? (Implementation Steps)
Sign up, Login, Logout feature (Account Authentication)	To allow session handling of users while using the app, and additionally record/store the needed info for attendance marking. This additionally allows auditing/tracking of user activities in case of anonymous access to the app, and to allow data integrity inside the database through tracked info.	Sign up: Implement an account creation page that the user can use to make an account using their info, involving their name, username, account role type, grade level, and section (if applicable), and password. After validating data, store it in the appropriate database table. Login and Logout: Implement a login page asking the user to enter their account username and password. If the account username and password match (checked via a SQLite query for finding the account), store the authenticated name, surname, ID, and account role while logged in. Clear the session info upon logout.
Role-based access control and attendance (Teachers, Students)	1. Prevent unauthorized access to certain	In the SQLite database schema, add a separate table for teachers and

	app features. 2. To segregate the SQLite database schema for account types, which simplifies queries and reduces overhead effects for large numbers of users	students. Then, in the authentication logic service layer, after the signup data was validated, insert the data into its appropriate SQLite table depending on the account role. The account role can be chosen during sign-up Form. Upon login with the account, based on the account role, show only the appropriate features/content to avoid unauthorized feature access.
Attendance features: Create Attendance, Attendance Dashboard.	Provides organized attendance management involving attendance sessions and records, specifically through the structured categories and the needed info of student attendance.	In the attendance dashboard, create a “Create Attendance” form that involves inputting the grade level and section, subject, date, and the start period relevant to the attendance. Additionally, in the attendance dashboard, create a table or a list of the attendance the teacher has made. The teacher can access the attendance and manage the attendance records of the students through a mini form that asks for the student ID and their punctuality status.
Data export of attendance records to CSV (Comma Separated Values)	Allows data portability outside of the SQLite Database. It additionally allows compilation and ease of access, given that it is a CSV.	Use an SQLite query to fetch attendance sessions, attendance records, and relevant student info such that the attendance sessions. Then, use the pandas library to convert the query results into a CSV file.

Pop-up messages for user actions in connection with event-driven GUI	Provide feedback and clarity on user actions, as well as inform the user of any errors in their inputs or actions and how to address them.	The UI Layer will read inputs and send them to the backend layer for input processing results. If the backend layer returns an error status of true, use a pop-up helper function and additionally use the returned error title and message. If there are no errors, give the user a pop-up of their successful user input (i.e, their input follows the happy path). Note: tkinter's messagebox was used for the popups. Moreover, the pop-up format mostly includes the type of error, what happened, and how to fix it.
System Settings (Attendance and Accessibility Settings)	Attendance Settings (Delete Attendance): To remove outdated/irrelevant attendance sessions and records Accessibility: To consider inclusion in UI design, and improve the usability of the app for those with disabilities	Delete All Attendances: Check first whether the user is a Teacher, as this is to prevent students from deleting attendance without notice. Then, ask the user to confirm their action. If they confirm their action, fetch the teacher's account ID, and delete all attendance sessions and records associated with the account ID. Accessibility: High Contrast Mode: Upon click, configure the app appearance such that the background does not blend in with the foreground/info needed by the user. Additionally, whenever the user's cursor hovers/selects an interactable widget, ensure that the focus color does

		not blend in with the info/widget.

2. Technologies Used (with justification)

2.1 Python

Python is an object-oriented programming language that is used in the project's code for the following reasons:

- Python's Self-documentation / Readability:

One of Python's main advantages is that it emphasizes and helps with the readability of code through simple syntax (syntaxes are self-descriptive and are English-like), minimalist design (does not use numerous parentheses or special characters), use of whitespace and mandatory indentation, and explicit, clear error messages (uses carets to pinpoint an issue with a line of code). Because of these reasons, it helps with the project code's maintainability and collaboration (In increased size and complexity of codes, reading code is done more than writing it. This helps with coders and developers when updates or snippets of code are given, as they have readability.

- Python's Comprehensive Standard Library:

Python has numerous libraries used for specific tasks such as computations, datetime handling, UI, string handling, etc. This helps with the project's code through the multiple libraries' readable and straightforward use and syntax, which decreases the complexity of code in certain areas of the program.

Though Python has a slow execution speed compared to other object-oriented programming languages (e.g, C++, Java) and high memory usage, Python's library and syntax allow the program to be written with fewer lines of code to achieve the same result (so long as the code is not messy and well-written).

2.2 Tkinter

Tkinter, additionally called "Tk interface," is a built-in library for creating a GUI (Graphical User Interface) for Python programs. The following is a list of reasons why Tkinter is used in the project code:

- No Separate installation required:

Tkinter is additionally installed in most Python installations. This means that no separate or complex installation is required to use it when using it in code. This helps in the project code by not making the code too heavily focused on making the user interface itself

- Tkinter's Syntax and learning curve: The syntax of tkinter is straightforward (self-documenting and self-explanatory), resulting in a less steep learning curve and helping as a less complex and introductory-like GUI for beginners or those who aren't familiar with implementing GUI in coding. This helps in the project code because it functions as a less complex way to establish how the UI for the program is intended to appear.

2.3 bcrypt

bcrypt is a type of password-hashing function that uses the Blowfish cipher for its algorithm. It uses a one-way, irreversible, slow-running password hashing function used specifically for secure password hashing and storage that can resist brute-force attacks (guessing all combinations, dictionary attacks) through its slow and memory-intensive hashing. It was used in this program to encrypt the passwords used for accounts and attendance (If the user were to gain access to the database or info about the passwords, they would be hashed and salted, so they can't decipher or decode them to find the account passwords of other users).

2.4 SQLite

SQLite is used in the following program for the following reasons:

- SQLite organizes the data stored into a structured table with columns that support different data types. This structure of storing data requires less complexity for data parsing and interpretation when fetched.
- SQLite databases have scalability when handling hundereds ot thousands of data, and can use indexes for speeding up data retrieval in large amounts of data. Additionally, because of its use of SQL query language, it allows multiple types of operations for data access and manipulation, such as joining, filtering, sorting, etc.
- SQLite's Data storage can handle corruption errors by reporting an error code (SQLITE_CORRUPT) instead of crashing or writing a partial line of code

2.5 tkCalendar

Tkcalendar is a Python module that allows a user to select certain dates via an interactive calendar widget. The main reason why it was used in the code program was to improve the UX (user experience) of creating attendance, as the calendar widget adds visual intuitiveness and interface cues for selecting dates. On a separate side note, the tkCalendar can adapt to different types of date formats when used, and it auto-fills / limits certain dates on the selector.

2.6 pandas

pandas is an open-source library module that is not part of Python's standard library. The main reason for its use, specifically for the export to CSV feature, was the library's ability to read and write from external data structures in diverse formats, which helps with the ease of implementation of the CSV Export feature.

2.7 pytest

Pytest is an open-source testing framework that makes automated unit testing simple in terms of syntax. The main reason it was used in the code was to quickly write and run tests for the app, so any failed tests that indicate an error in the implementation of features or code blocks can be identified.

Unit testing was used for the testing method of the code program, specifically for the validator functions. The tests were used to determine if the functions would return the expected output given the simulated inputs (happy path-wise or not)

Backend-frontend Communication

3.1 Tkinter (Frontend)

Note: UI means user interface, and UX means user experience

Tkinter, as the frontend and graphical user interface, contributes to the project code through its interactions with the backend, Python, and with its widgets and pop-up messages (that give info about the user's actions, such as errors or info notices), which provide the user interactivity rather than using a text-based user interface. From a UI and UX point of view, it displays widgets such as labels, buttons, entries, and tables in a

vertical format that allows users to navigate the different parts of the program. For its interactions with the backend, when users interact with widgets, Tkinter takes this input and gives it to the backend of the program, and changes the current frame shown to the user accordingly (or gives the user pop-up messages). For example, when the user enters login credentials in the entry fields and presses the "Log In" button, Tkinter interacts with Python by going to the corresponding function. Then it gives Python the entered info and gets the computed results or info. After that, tkinter gives or changes the respective UI based on the computation given.

3.2 Python (Backend)

Python, as the backend and logic flow of the program, contributes to the project code by determining how the program works. In addition, it acts as the middle with its interaction with the frontend and storage layer. It performs certain processes such as input validation, data integrity, computation, and more. Whenever the backend receives an input and/or an interaction from the frontend, the backend generally first does input validation to check if the user input meets certain conditions. After that, the backend does computation for certain actions or calculations, such as hashing, time parsing, or time calculation. After that, it either results in the backend interacting with the frontend to do or display certain widgets, or it interacts with the storage to store certain data from its computation. To add on, the backend can communicate with the storage the retreive certain data from tables, compare it with the data given from the frontend. This results in either interaction with the frontend or the storage.

3.3 SQLite (Storage)

SQLite, as the storage of the program, contributes to the project code by being able to save certain data inputs that result from the backend processing it. Then, it stores that data in certain tables of the attendance database. In addition, the attendance database can retrieve data from its tables to give to the backend for computing. The storage layer does not directly interact with the frontend. Instead, it uses the backend and SQL commands to communicate with the backend to process it and display or use it for the frontend. An example would be when the user goes to the view attendances, and the storage gives the appropriate table data to the backend for processing. Then the backend gives the frontend the processed data to output it.

4. Key Design Decisions or Trade-offs

4.1 Pop-up validation messages

For informing the user of info, warnings, and questions, a pop-up validation message was used. Pop-up messages assist in the user experience of the program, as the flow of the application is an event-driven GUI. Additionally, it reduces the confusion of why certain actions don't result in the expected outcome (error messages). Pop-up messages assist this by displaying the error to the user instead of appearing as a console message. Pop-up validation messages have trade-offs, such as their sudden disruption, reducing focus, and repetitiveness for certain actions in the program. However, these trade-offs were accepted in the design choice because it provides user feedback on whether something went wrong, and it provides info in the case that their input results in an error, along with instructions on how to prevent it from constantly appearing.

4.2 Database Usage (SQLite)

The project code uses SQLite for storing data on what actions were done during a user session. This database was chosen mainly because SQLite does not require numerous steps for the server process or setup. Moreover, SQLite uses a serverless architecture for its database engine. The serverless architecture contributes to the project code by having no server process for installation, configuration, or starting. The approach of using SQLite allows the program to store and retrieve user accounts, attendance records, and submissions. To add, SQLite supports full ACID (Atomicity, Consistency, Isolation, Durability) transactions, helping with the data integrity of the program by not inserting corrupt or malformed data entries in the database in the scenario of system crashes.

Using SQLite has tradeoffs such as limited concurrent access handling and scalability for large user environments, which other database systems (e.g., MySQL, PostgreSQL) can provide. These tradeoffs were accepted in the design choice because it eliminates the requirements for networked or high-concurrency database access. To add, it is acceptable given that the project scope and expected usage scale are expected to be in the hundreds or thousands range (if not tens or hundreds of thousands of users)

4.3 Exception-based attendance marking

For the attendance creation and marking, it utilizes exception-based attendance management, meaning that it assumes the “ideal” scenario for an attendance session (i.e, everyone is present), and only asks the user to input any exception that deviates from that scenario (e.g, someone who is tardy). Although it doesn’t

5. References to Ethical Considerations in Programming Choices

5.1 Data protection:

The program system implements hashing and salting of account passwords by using the bcrypt(UTF-8) library instead of storing credentials in plaintext. This design decision was chosen to reduce the effects of data breaches and credential exposure of accounts and attendance in the scenario of unauthorized database access. The structure works by hashing and salting user credentials when inserted into the account tables of the attendance database. To add on, sensitive data is only used in backend-storage interactions, ensuring that sensitive data will not reach the frontend. It is additionally noted that the storage layer uses parameter placeholders to prevent unauthorized access to attendance.db through SQL injections. Though this overall approach for user privacy does not use network-level security measures, the current security measures are appropriate for a local and non-networked application.

5.2 Pop-up Messages:

Although pop-up messages are designed to help users understand what actions happened based on their inputs, pop-up messages, especially for tkinter, have experienced technical challenges in terms of screen reader problems. This is because the text from the messages appears without introducing itself to the system screen reader first, which results in the message not being read, and can confuse users who depend on the screen reader. Despite this, the program can mitigate the effects by putting focus on the pop-up message. Though it doesn’t completely mitigate the screen reader problem, it at least helps improve the accessibility of the code app despite tkinter’s historical issue about accessibility support